MODULE 05 · DOMAIN 1 - SDLC AUTOMATION

Artifacts and Images

Where build output lives: S3 artifact stores, CodeArtifact packages, ECR images, and golden AMIs from EC2 Image Builder.

DOMAIN 1 of 6 EXAM WEIGHT 22% MODULE 5 of 34

admin@sthithapragna.com

An artifact store answers three questions: **who can fetch it**, **how long it lives**, and **whether it can change**.

admin@sthithapragna.com

Four kinds of artifact, four homes

ARTIFACT	HOME	CONSUMED BY
Pipeline hand-offs	S3 artifact store	The next pipeline action
Libraries and packages	CodeArtifact	Other builds, through npm, pip, Maven
Container images	Amazon ECR	ECS, EKS, Lambda, App Runner
Machine images	AMIs, from EC2 Image Builder	Launch templates and Auto Scaling

ON THE EXAM

'Packaged code must be an upstream source for future builds' means CodeArtifact. An S3 bucket is not a package repository.

The pipeline's S3 artifact store

- One artifact bucket per Region the pipeline acts in. The console can create it for you.
- Encrypted by default with the AWS managed key for S3.
- Cross-account actions need a **customer managed key**, as in module 1.
- Artifacts accumulate forever unless an **S3 Lifecycle** rule expires old ones.

> ON THE EXAM
>
> Artifact bucket growing without limit: an S3 Lifecycle expiration rule. No Lambda clean-up job.

A CodeArtifact domain

pypi-store
External connection: PyPI
Caches public packages

npm-store
External connection: npmjs
Caches public packages

team-web
Upstream: shared, npm-store
What developers point at

Domain: acme
One KMS key for everything
Domain policy: which accounts
Deduplicated storage
Assets stored once, shared by every repository.

team-data
Upstream: shared, pypi-store
Another team's view

shared
Internal libraries
Published by CI only

ON THE EXAM

Teams point at one repository. Upstreams make internal and public packages appear together.

CodeArtifact rules worth knowing

- Formats: npm, PyPI, Maven, NuGet, Swift, Ruby, Cargo, and generic.
- A repository can have several **upstreams**, searched in order, but at most one **external connection**.
- Package versions fetched from upstream are **retained**, so builds survive a public package being deleted.
- Publishing a new version emits an **EventBridge** event that can start downstream pipelines.

> ON THE EXAM
>
> Rebuild every consumer when a shared library publishes a new version: EventBridge on the CodeArtifact event.

Reaching CodeArtifact

- Clients use a temporary **authorisation token**: up to 12 hours, 12 by default after login.
- Getting a token needs codeartifact:GetAuthorizationToken **and** sts:GetServiceBearerToken.
- Other accounts are admitted by the **domain policy** and the **repository policy**.
- From a VPC with no internet: interface endpoints **codeartifact.api** and **codeartifact.repositories**, plus an **S3 gateway endpoint** for assets.

> ON THE EXAM
>
> Private build reaches CodeArtifact metadata but not packages: the S3 gateway endpoint is missing. Python and Swift skip it.

Dependency confusion, and how to stop it

- An attacker publishes a **public** package with your internal package's name and a higher version.
- A resolver that merges internal and public sources may pick the attacker's version.
- **Package origin controls** set, per package, whether new versions may be published directly and whether they may arrive from upstream.
- **Package groups** apply those controls by pattern, such as every package in your npm scope.

> ON THE EXAM
>
> Internal names must never be satisfied from a public registry: block upstream for that package group.

ECR fundamentals

- One private registry per account and Region, containing repositories of images.
- A **tag** is a movable label. A **digest** is the image's content hash and never changes.
- **Tag immutability** stops a tag being overwritten. Since July 2025, **exclusion patterns** can exempt tags such as latest.
- Encryption with AES-256 by default, or a KMS key chosen when the repository is created.

> ON THE EXAM
>
> 'Guarantee a deployed tag always means the same image': tag immutability, or deploy by digest.

An ECR lifecycle policy

```
1  { "rules": [
2    { "rulePriority": 1,
3      "description": "Untagged images older than 7 days",
4      "selection": { "tagStatus": "untagged",
5        "countType": "sinceImagePushed", "countUnit": "days", "countNumber": 7 },
6      "action": { "type": "expire" } },
7    { "rulePriority": 2,
8      "description": "Keep the 30 most recent release images",
9      "selection": { "tagStatus": "tagged", "tagPatternList": ["v*"],
10       "countType": "imageCountMoreThan", "countNumber": 30 },
11     "action": { "type": "expire" } } ] }
```

4-5 By age: untagged images expire a week after push.

9-10 By count: beyond 30 v-tagged images, the oldest expire.

ON THE EXAM

Rules only ever select images to expire. Lower rulePriority is evaluated first. Preview a policy before applying it.

Getting images into other Regions and accounts

MECHANISM	WHAT MOVES	BEST FOR
Replication	Every new matching push	DR; Regions running everything
ECR-to-ECR cache	Only images actually pulled	Many Regions, few images each
Public pull cache	Docker Hub, GHCR, Quay images	Rate limits; scanning
Cross-account pull	Nothing: a policy grants it	Several accounts, one registry

ON THE EXAM

Replication copies new pushes only, never existing images. Pull through cache fetches on first pull.

Repository creation templates

- ECR creates repositories for you during **pull through cache**, **replication** and **create on push**.
- A template, matched by prefix, sets their **KMS key**, tag immutability, lifecycle policy, repository policy and tags.
- Templates apply only when a repository is **created**. Existing repositories keep their settings.
- Encryption cannot be changed later, so a non-compliant repository must be deleted and recreated.

> ON THE EXAM
>
> Automatically created repositories must use a specific KMS key: a repository creation template for that prefix.

Proving where an image came from

- **ECR managed signing** (November 2025) signs images automatically on push, using AWS Signer.
- Signatures are stored beside the image as OCI referrers. Notation can sign client-side instead.
- Verification happens at deploy time, for example by an admission controller on EKS.
- For Lambda zip packages, **code signing** with AWS Signer blocks unsigned code from deploying.

> ON THE EXAM
>
> Only run images or code from trusted sources, with least effort: managed signing plus verification at deploy.

A golden AMI pipeline with EC2 Image Builder

01

Recipe

Base image plus build components: hardening, agents.

02

Build and test

Test components prove the image works.

03

Scan

Amazon Inspector checks for vulnerabilities.

04

Distribute

Copy to Regions, share with accounts, update launch templates.

05

Retire

Lifecycle policies deprecate and delete old images.

ON THE EXAM

Scheduled weekly, or whenever the base image updates. The same service builds container images too.

Bake it in, or bootstrap at launch?

BAKE

Everything in the AMI

- Instances start ready, in seconds
- Every instance identical to the image
- Changes need a new image
- Best for fast scale-out

BOOTSTRAP

Configure at launch

- User data or scripts install software
- Flexible, no rebuild to change
- Slow, and fails when repositories do
- Instances can drift apart

THE TIEBREAKER

Scale-out is too slow because instances install software at boot: bake a golden AMI. Keep only config at launch.

A company publishes internal npm packages under its @acme scope to a CodeArtifact repository. That repository has an upstream repository with an external connection to the public npm registry.

A security review finds that anyone could publish a public package named @acme/auth-utils with a higher version, and builds might resolve it. Public packages must remain available.

Which solution prevents this with the LEAST operational overhead?

A. Remove the external connection so that no public packages can be fetched

B. Pin exact versions of every dependency in each project's lock file

C. Create a package group for the @acme scope with origin controls that block upstream versions

D. Move the internal packages into a separate CodeArtifact domain

◆ PAUSE AND DECIDE

Which solution prevents this with the LEAST operational overhead?

A Remove the external connection

Stops the attack by breaking every public dependency. Violates the stated requirement.

B Pin every version

Helps existing dependencies, but every new or updated dependency is exposed again. Ongoing effort.

C Package group blocking upstream

Internal names can only come from direct publishing. Public packages still flow. One configuration.

D Separate domain

Projects still need both sources merged, so the confusion simply moves. A migration for nothing.

THE SIGNAL

Internal name, public impostor: package origin controls, applied by pattern with a package group.

A company pushes several hundred container images to Amazon ECR in us-east-1. It runs workloads in six other Regions, but each Region runs only a handful of those images.

The company wants in-Region image pulls for faster start-up, with the lowest storage cost and the least operational overhead.

What should the company configure?

A ECR-to-ECR pull through cache rules in each of the six Regions, with us-east-1 as upstream

B Registry replication from us-east-1 to all six Regions

C A pipeline step that copies each new image to all six Regions

D Point every Region's workloads at the us-east-1 repositories directly

◆ PAUSE AND DECIDE

What should the company configure?

A ECR-to-ECR pull through cache

Each Region stores only the images it actually pulls, then pulls them locally. Managed sync.

B Replication to all six

In-Region and managed, but stores every image in every Region. Six times the storage for images never run.

C Pipeline copy step

Same storage problem as replication, plus custom work to maintain.

D Pull from us-east-1

No extra storage, but every pull is cross-Region. Fails the in-Region requirement.

THE SIGNAL

Many Regions, few images each: pull through cache. Every Region needs everything: replication.

A company patches its EC2 Auto Scaling fleets by hand once a month. The fleets run in three accounts and two Regions, and scale-out is slow because instances install agents at boot.

The company wants a hardened image rebuilt weekly with the latest patches, scanned for vulnerabilities, made available in every account and Region, with launch templates updated automatically.

Which solution meets these requirements with the LEAST operational overhead?

A. A scheduled Lambda function that runs Systems Manager Automation to patch and share an AMI

B. Packer in a weekly CodeBuild project, with scripts that copy and share the AMI

C. Launch template user data that installs agents and applies patches at every boot

D. An EC2 Image Builder pipeline on a weekly schedule, with Inspector scanning and distribution settings for accounts, Regions and launch templates

◆ PAUSE AND DECIDE

Which solution meets these requirements with the LEAST operational overhead?

A **Lambda plus Automation**

Workable, but hand-builds scheduling, scanning and distribution that Image Builder includes.

B **Packer on CodeBuild**

A real pattern, but the copying, sharing and template updates are scripts you maintain.

C **Patch at every boot**

Makes scale-out slower still, and instances drift. The opposite of a golden image.

D **Image Builder pipeline**

Schedule, build, test, scan, distribute and update launch templates are all built in.

THE SIGNAL

Build, scan, distribute across accounts and Regions, update launch templates: every word is an Image Builder feature.

An auditor requires that release tags in an ECR repository, such as v2.4.1, can never be overwritten. The team's pipeline also moves a staging tag to each new build.

When the team enabled tag immutability, the pipeline started failing because the staging tag could no longer be moved.

What should the team do with the LEAST effort?

A Disable tag immutability and review release tags in a monthly audit

B Move staging images into a separate repository that has mutable tags

C Keep tag immutability and add an exclusion pattern for the staging tag

D Stop using tags and change every deployment to reference image digests

◆ PAUSE AND DECIDE

What should the team do with the LEAST effort?

A Disable, audit monthly

Replaces a preventive control with a detective one. The auditor asked for 'never'.

B Separate repository

Works, but duplicates images, policies and pipeline logic across two repositories.

C Exclusion pattern for staging

Release tags stay immutable; the one moving tag is exempt. A single setting since July 2025.

D Digests everywhere

Good practice, but a change to every deployment. Far from least effort.

THE SIGNAL

Most tags must be fixed, one or two must move: immutability with exclusion patterns.

Module 5 in five lines

- S3 for pipeline hand-offs, CodeArtifact for packages, ECR for images, Image Builder for AMIs.
- CodeArtifact: upstreams in order, one external connection, origin controls stop dependency confusion.
- ECR lifecycle rules only expire. Replication copies all new pushes; pull through cache copies what is pulled.
- Repository creation templates set KMS keys and policies on repositories ECR creates for you.
- Bake golden AMIs for fast scale-out. Image Builder schedules, scans and distributes them.

ON THE EXAM

Next, module 6: CodeDeploy on EC2 and on-premises servers. Agents, the appspec, deployment configurations and rollback.